임베디드 SW 포트폴리오

조경훈

철도 차상 시스템 임베디드 SW 개발 · 지에이티(GAT) 연구소 2025.12 ~ 현재

C · C++ | UART · RS-232/485 · CAN · Ethernet(UDP · lwIP) | Bare-metal · Embedded Linux

E-mail rudgns4222@naver.com

기술 스택 및 프로젝트 요약

소속 지에이티(GAT) 연구소 | **재직** 2025.12 ~ 현재 | **담당** 철도 차상 시스템(차량 제어·방송) 임베디드 SW 개발·검증

기술 스택

언어	C · C++
타깃 · OS	Cortex-M4/M7 베어메탈 (STM32 HAL) i.MX8M Plus · QNX 7.1 (크로스 빌드·배포) Cortex-A72 SoC · Linux 5.4 (드라이버 분석)
통신	UART · RS-232 · RS-485 · SPI · CAN Ethernet: UDP 소켓 · lwIP TRDP · MVB (구조 분석)
디버깅 · 계측	Visual Studio 디버거 · Wireshark ftrace · Fault Analyzer 오실로스코프 · Logic Analyzer CAN 분석기 · 멀티미터
빌드 · 배포	STM32CubeIDE · QNX SDP(qcc) · WSL · Git u-boot · TFTP · JTAG · ST-Link

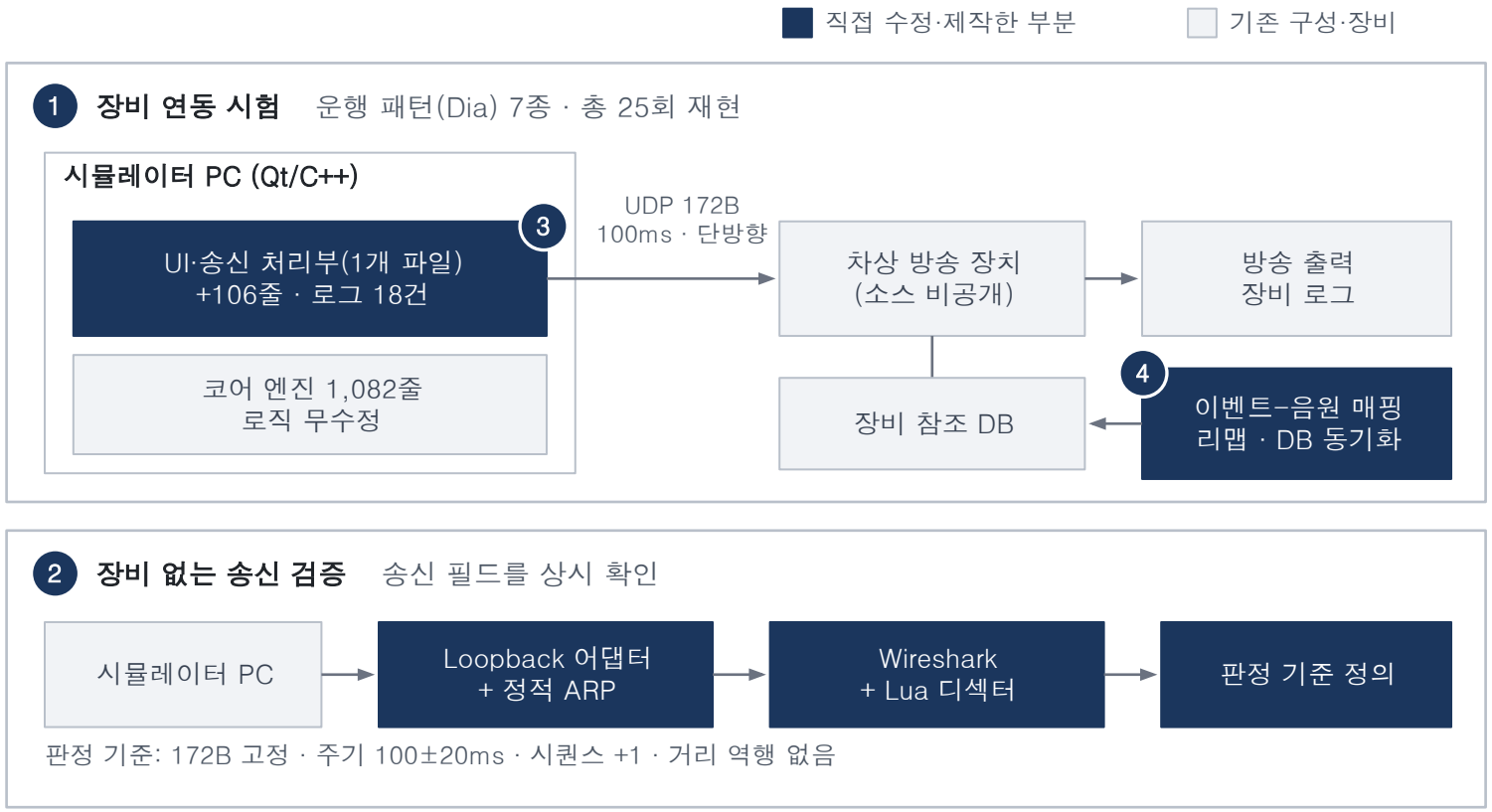
프로젝트 요약

프로젝트	기간	역할	핵심 결과	페이지
홍콩 MTR 동철선 차상 방송 (PA/PIS) SW 이슈 대응	2026.02 ~ 현재	사내 4인 · 시뮬레이터 디버깅 실무, 회의록 작성 · 현지 시험 참여	보고 이슈 3건 수정, MTR 검토 Pass	3–4
개발 보드 통신 인터페이스 브링업·검증	2026.05 ~ 07	단독 · Cortex-M4/M7 펌웨어, 계측 검증	CAN 500kbps 송수신, lwIP Ping 정상 응답	5–7
인도 Vande Bharat TCMS 차량제어장치(CC)	2026.08 ~ 현재	팀 5인 · 빌드 환경 구축, 펌웨어 분석, 보드 업데이트, 매뉴얼 작성	QNX 빌드 환경 구축, CPU 보드 20대 설정, 업데이트 매뉴얼 작성	8
[인턴] 리눅스 커널 UART 드라이버(PL011) 분석	2024.10 ~ 12	텔레칩스 BSP 인턴 · 개인 과제(멘토 지도)	함수 호출 경로 정리, 수신 에러 4종 검출	9–10

도식 색 구분 (전 페이지 공통) ■ 직접 작성·수행한 부분 □ 기존 구성·장비

방송 시뮬레이터 디버깅 및 송신 검증

기간 2026.02 ~ 현재 | **팀** 사내 4인 | **역할** 시뮬레이터 디버깅 · 고객 대응 | **환경** Qt 5.7 · C++ · UDP · Wireshark



인사이트 장비 내부 확인이 불가하여 송신 데이터를 장비 없이 먼저 검증
→ 송신이 정상인 상태에서 남은 현상은 장비·데이터 쪽 문제로 분리

목표

시행청(MTR)이 보고한 방송 오동작의 재현·해소와
수정본 검증

수행 내용

1. **재현** 운행 패턴 7종으로 총 25회 재현, 이슈별 발생 조건 정리
2. **검증 환경** 장비 없이 송신 필드를 상시 확인하는 환경 구축
3. **수정** UI·송신 처리부 1개 파일만 수정(+106줄), 코어 엔진 무수정
4. **데이터** 이벤트–음원 매핑 리맵, 시뮬레이터·장비 DB 동기화
5. **고객 대응** 현지 야간 본선 주행 시험 참여·결과 기록, 격주 화상회의 회의록 17건 작성, 방송 누락 건 영문 분석 보고서

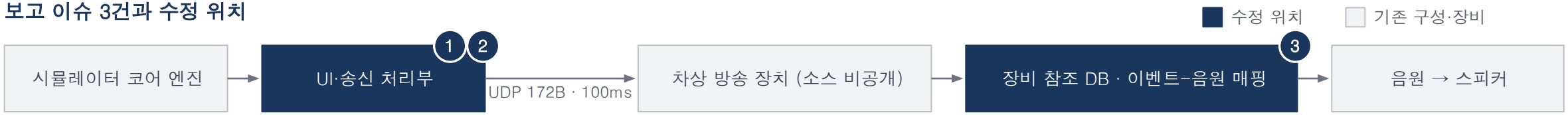
결과

- **2026.05** 보고 이슈 3건 수정본 송부
- **2026.06** MTR 워크숍 시험에서 기존 현상 해소 확인
- **2026.07** MTR 검토 특이사항 없음(Pass)

고객사 이슈 3건 SW 디버깅

기간 2026.02 ~ 현재 | **팀** 사내 4인 | **역할** 시뮬레이터 디버깅 · 고객 대응 | **환경** Qt 5.7 · C++ · UDP · Wireshark

보고 이슈 3건과 수정 위치



이슈	재현	원인	수정
① 시작 시 이전 역 기준 방송	25회 중 23회	STOP 처리부가 역 인덱스, 거리를 0으로 되돌린 뒤 송신 → 다음 START의 기준 컨텍스트가 오염 (Visual Studio 중단점으로 버튼 처리 흐름 추적)	**리셋 블록 제거: STOP은 타이머만 정지하고 역·거리 보존**
② 재시작 시 출발(+200m) 방송 미발화	25회 중 14회 장비에서만 발생, 루프백 송신 필드는 정상	원인 후보 2개(설정 패킷 유실 / 장비 처리 지연), 장비 내부 비공개로 특정 불가	**START 시 같은 패킷 2회 송신 후 주기 시작, 거리는 1m 단위로 갱신**
③ 특정 역 도착 방송 음원 미출력	25회 중 3회 이벤트는 발화, 음원만 미출력	장비가 태그 달린 음원명에 접미(:1)를 붙여 검색 → 해당 음원이 장비에 없어 미출력 (정상·오류 로그 대조로 역추적)	**이벤트–음원 매핑만 리맵(SQL), 시뮬레이터·장비 DB 동기화**

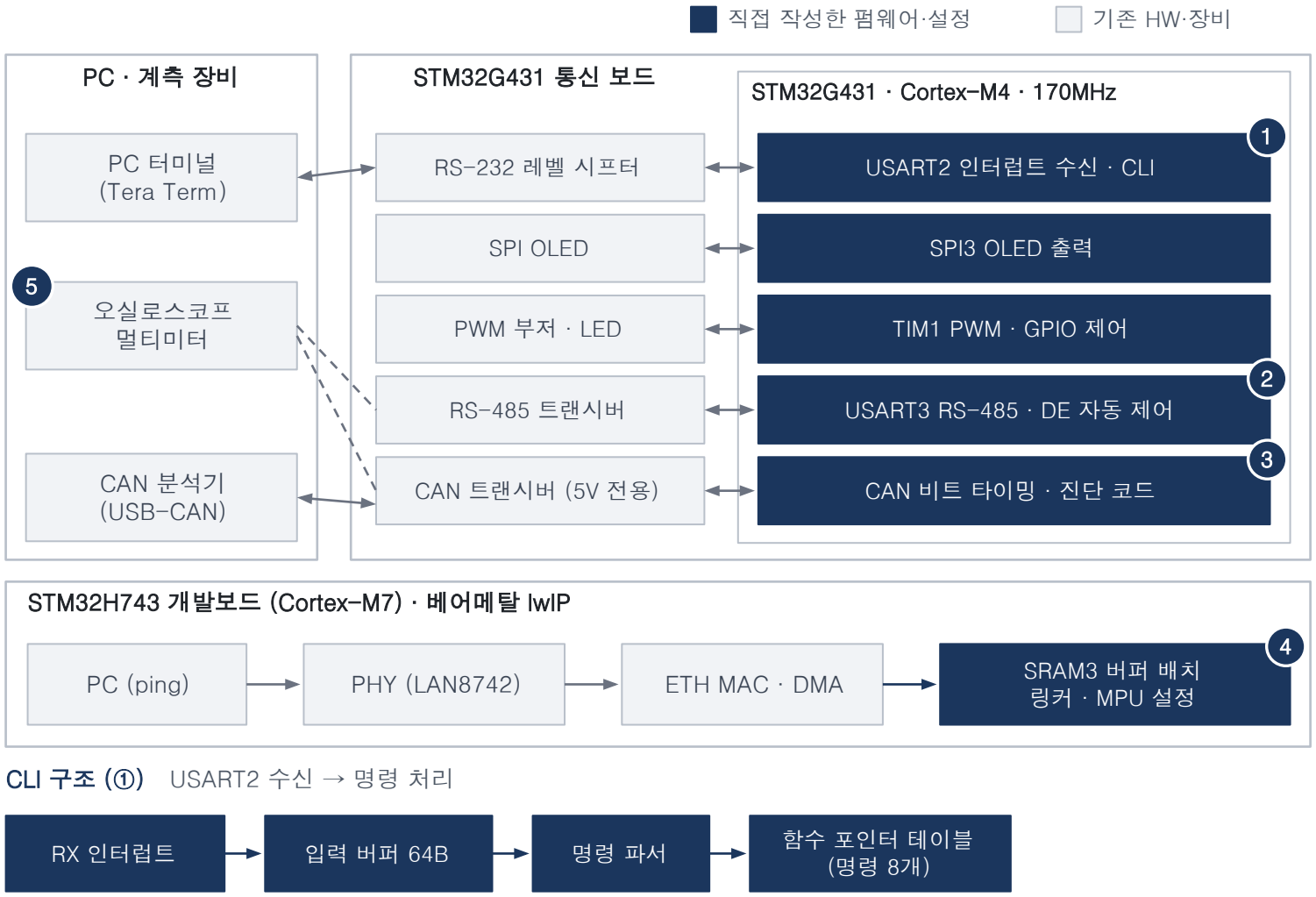
회귀 시험 기준 테스트 케이스 15건 · 합격 조건 6개 | **2회 송신 대기** 1차 송신 후 100ms · 2차 송신 후 50ms, 장비 연동 시험으로 결정

결과

2026.06 보고 이슈 3건 수정본 송부 → **2026.06** 고객사(MTR) 자체 시험에서 해결 확인 → **2026.07** MTR 최종 검토 특이사항 없음(Pass)

STM32 통신 인터페이스 브링업 및 계측 검증

기간 2026.05 ~ 07 | **팀** 단독 | **역할** 펌웨어 브링업 · 통신 계측 검증 | **환경** STM32G431 · STM32H743 · STM32CubeIDE · HAL



CLI 구조 (①) USART2 수신 → 명령 처리

목표

사내 프로젝트 보드와 같은 계열 MCU로 통신 인터페이스를 직접 구현해 선행 검증

수행 내용

1. **연동** UART → SPI OLED → PWM → CLI 순 단계별 연동
2. **구현** RS-485 반이중, DE 하드웨어 자동 제어
3. **설계** CAN 비트 타이밍, 에러 카운터·상태 진단 코드
4. **브링업** STM32H743 lwIP, SRAM3 배치·MPU 설정
5. **계측** 오실로스코프·멀티미터·CAN 분석기로 검증

결과

- **CAN 500kbps** 양방향 송수신, 비트폭 2µs 측정
- **RS-232 · RS-485** 115200bps 송수신
- **lwIP Ping** 정상 응답

CAN 미수신 원인 추적 및 비트 타이밍 검증

기간 2026.05 ~ 07 | **팀** 단독 | **역할** 펌웨어 브링업 · 통신 계측 검증 | **환경** STM32G431 · STM32H743 · STM32CubeIDE · HAL

추적 순서

■ 단서·원인 □ 확인·배제

내부 루프백 경로 정상 → STB 핀 정상 레벨 → 설정 대조 500kbps · 120Ω 일치 → 진단 코드 수신 0건 Bus-Off → 도통 측정 배선 교차 도통 → 회로도 추적 5V 레일 미공급

전원 경로

전: ST-Link 3.3V → 강압 레귤레이터 ✕ CAN 트랜시버 (5V 전용)

후: USB-C 5V → CAN 트랜시버 (5V 전용) → CAN 버스 송수신

비트 타이밍 설계

170MHz ÷ 17 → tq 100ns
Sync 1 + Seg1 15 + Seg2 4 = 20tq
20tq = 2µs → 500kbps, SJW 2



인사이트 에러 카운터·상태 출력 전에는 SW 설정만 점검,
진단 코드 추가 후 Bus-Off 확인 → 배선·전원 등 물리 계층 점검으로 전환
→ 통신 드라이버 작성 시 실패 상태를 출력하는 진단 경로를 함께 구현

현상 CAN 분석기 채널끼리는 통신 정상
보드만 프레임 수신 0건

해결 USB-C로 5V 공급
분석기 2채널 동시 결선을 단독 결선으로 정정

결과 양방향 송수신 확인
비트폭 2µs 측정, 설계값(500kbps)과 일치

CAN 차동 신호 파형 (500kbps / 비트폭 2µs)



lwIP HardFault 분석 및 힙 재배치

기간 2026.05 ~ 07 | **팀** 단독 | **역할** 펌웨어 브링업 · 통신 계측 검증 | **환경** STM32G431 · STM32H743 · STM32CubeIDE · HAL

추적 순서

■ 원인·조치 □ 확인·변경 전

Fault Analyzer
CFSR 0x400
IMPRECISERR

→

콜스택
수신 콜백 pbuf
포인터 무효 주소

→

D-Cache OFF
문제 지속
→ 캐시 외 원인으로 판단

→

메모리 배치
힙 시작 주소
= 디스크립터 주소

SRAM3 배치

0x30040000 ~ 0x30047FFF · 32KB · MPU Normal Non-cacheable



현상 Ping 시험 중 HardFault
CFSR 0x400 (IMPRECISERR)

원인 lwIP 힙 시작 주소(0x30040000)가 DMA 디스크립터 영역과 겹침 → 원인으로 판단

해결 힙을 디스크립터·수신 버퍼 뒤 0x30045000으로 이동, 관련 설정 정리

결과 HardFault 없이 Ping 정상 응답

인사이트 폴트 레지스터(CFSR) 기반 디버깅 절차
→ DMA 버퍼와 힙이 같은 SRAM을 쓸 때는 시작 주소·크기를 대조해 영역 겹침 방지

TCMS 빌드 환경 구축 및 펌웨어 분석

기간 2026.08 ~ 현재 | **팀** 5인 | **역할** 빌드 환경 구축 · C 펌웨어 분석 · 보드 펌웨어 업데이트 · 매뉴얼 작성 | **환경** QNX 7.1 · i.MX8M Plus · C · pthread

■ 직접 구축·수행 □ 기존 시스템(분석·문서화 대상)

1 빌드 환경

WSL + QNX SDP 7.1 qcc(aarch64le) 크로스 빌드 → CC 실행 바이너리 → i.MX8M Plus CPU 보드 QNX 7.1

2 CC 펌웨어 구조 (소스 분석)

main 루프	TRDP 송신 스레드	UDP 송수신 스레드 2	MVB 폴링 스레드
DI 읽기 → 로직 → DO 쓰기	Master/Slave 중 택1	TC 프로세스와 교환	

기동 시 IP 옥텟으로 호차·Master/Slave 판별, 공유 데이터는 pthread mutex로 보호

→ **소스 분석 노트 작성**

통신 버스 (3중)

→ **통신 구조 문서 작성**

TRDP · Ethernet PD 멀티캐스트 100ms	CC · MC1 · TC1 · MC2 노드 (Master/Slave 이중화)
MVB TRDP 백업	제동 · 공조 · 출입문 · PIS 등 하부장치
CAN I/O 보드 연결	IO 보드 5종 AIO · DI · DO · SDI · SDO

3 보드 펌웨어 업데이트

FPGA 비트스트림 iMPACT · JTAG | QNX IFS 이미지 u-boot · TFTP | IO 보드 5종 펌웨어 ST-Link | CPU 보드 20대 IP · startup.sh 확인·교체

4 매뉴얼 작성 보드 업데이트 절차서 작성 → 협력사 매뉴얼로 전달, CPU 보드 IP·startup.sh 설정 현황 문서화

수행 내용

1. **빌드 환경 구축** WSL · QNX SDP 7.1 크로스 빌드 구성, 빌드 로그로 누락 헤더·설치 경로 확인, 3개 타깃 중 CC만 빌드 가능함을 확인
2. **C 펌웨어 분석** 메인 소스를 실행 흐름 기준으로 분석, 스레드 구성과 TRDP·MVB·CAN 통신 구조 정리 → 소스 분석 노트, 통신 구조 문서 작성
3. **보드 펌웨어 업데이트** FPGA(JTAG), QNX 이미지(u-boot·TFTP), IO 보드 5종(ST-Link) 업데이트, CPU 보드 20대 IP 확인, startup.sh 일괄 교체(TFTP)
4. **매뉴얼 작성** 보드 업데이트 절차서 작성 → 협력사에 매뉴얼로 전달, CPU 보드 20대 IP·startup.sh 설정 현황 문서화

결과

- QNX 크로스 빌드 환경 구성
- 소스 분석 노트 · 통신 구조 문서 작성
- CPU 보드 20대 설정, 업데이트 매뉴얼 협력사 전달

PL011 UART 드라이버 호출 경로 분석

기간 2024.10 ~ 12 | **소속** 텔레칩스 BSP 개발 인턴 | **형태** 개인 과제 · 멘토 지도 | **환경** Cortex-A72 SoC 보드 · Linux 5.4



목표

UART 드라이버와 커널 서브시스템 사이의 호출 흐름 파악

수행 내용

1. **구성** 내부 루프백에서 PC-보드 결선으로 변경하고 Logic Analyzer 연결
2. **작성** pthread 송신 · select() 수신 시험 프로그램
3. **설정** ftrace 필터: 부팅 bootargs, 쉘 스크립트
4. **추적** 부팅 초기화와 write·read 호출 순서를 다이어그램으로 정리

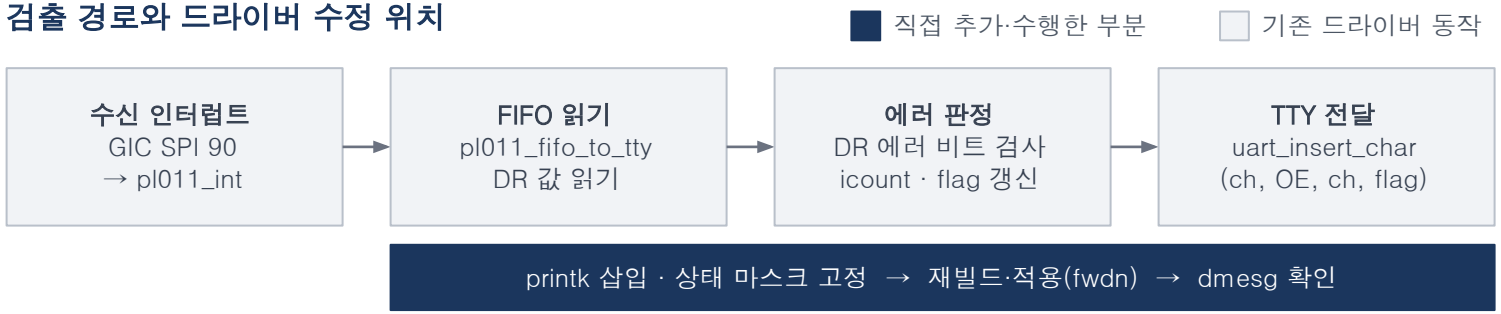
결과

- **부팅 초기화** AMBA Bus → PL011 → Serial Core → TTY 순 등록 확인
- **write** 시스템 콜에서 드라이버까지 호출 순서 확인
- **read** 수신 인터럽트가 버퍼를 채우고 read()가 읽어 가는 구조 확인 (드라이버 구간은 PC 송신 구성에서 추적)

UART 수신 에러 4종 유발 및 검출

기간 2024.10 ~ 12 | **소속** 텔레칩스 BSP 개발 인턴 (임베디드 스쿨 1기) | **형태** 개인 과제 · 멘토 지도 | **환경** Cortex-A72 SoC 보드 · Linux 5.4

검출 경로와 드라이버 수정 위치



에러별 유발 조건과 결과

에러 · DR 비트	유발 조건 (송신 → 수신)	로그 누적	TTY flag	파형 디코드
Framing · bit 8	115200bps → 9600bps	**Frame 734**	TTY_FRAME	framing error
Break · bit 10	9600bps → 921600bps	**Break 54,551**	TTY_BREAK	framing error
Parity · bit 9	패리티 Odd → Even	**Parity 5,386**	TTY_PARITY	parity error
Overrun · bit 11	11,200자 문자열 반복 송신	**Overrun 1**	별도 인자	–

드라이버 로그 발췌 · Framing 시험

타임스탬프·일부 행 생략

```
PL011:[JKH] Data read from FIFO: 0x10130    ← bit 8(FE) 세트 · 데이터 0x30
PL011:[JKH] Framing Error detected
PL011:[JKH] Inserting char to TTY: 0x130 (flag: 0x2)    ← TTY_FRAME
PL011:[JKH] Errors - Break: 91, Parity: 0, Frame: 734, Overrun: 0
```

0x10000은 드라이버가 수신 문자에 붙이는 더미 비트

목적 수신 에러가 드라이버에서 검출되어 TTY로 전달되는 과정을 레지스터 값으로 확인

방법 pl011_fifo_to_tty()에 printk 삽입
read_status_mask를 고정해 에러 비트 유지
송수신 조건을 바꿔 에러 4종 유발
로그와 Logic Analyzer 파형 대조

결과 4종 모두 드라이버 로그에서 검출
DR 에러 비트와 TTY flag의 대응 확인
Overrun은 문자 flag와 별도 인자로 전달

측정 Logic Analyzer로 보레이트 5종 × 1·32·320B 전송 오차율 측정: 0.0039~0.347%